

# Dashboard DeviceStatus.health-is-an-object regression guard — QA evidence

**Run-id:** 20260610T113752Z-dash-board-health-guard **Date (UTC):** 2026-06-10T11:38:28Z **Scope:** dashboard/ Vitest component layer **Constitution:** §11.4.135 (every fixed defect gets a permanent regression guard) · §11.4.115 (RED-on-broken-shape → GREEN-on-fixed-shape with a RED\_MODE polarity switch) · §11.4 anti-bluff (real Vitest runs captured in vitest\_run.txt).

## The product bug (closed by commit 4658125, shipped WITHOUT a guard)

commit 4658125 – "fix(dashboard): DeviceStatus.health is an object – fix Fleet-detail crash + cover"

The server wire shape of DeviceStatus.health is an **OBJECT** { ok: bool, last\_error\_code: string|null } (server internal/api/wire.go DeviceHealth). The dashboard wrongly typed it health?: string and the Fleet device-detail screen (FleetScreen.tsx DeviceDetail) rendered it as a bare React child:

```
<Row label="health" value={status.data.health ?? "-"} />
```

With the real object shape React throws “**Objects are not valid as a React child (found: object with keys {ok, last\_error\_code})**”, the <DeviceDetail> render throws, and the **entire device page blanks** for the operator (a §11.4.108 SOURCE→USER-VISIBLE failure). The fix added the DeviceHealth type and projects health.ok → an ok/unhealthy badge + health.last\_error\_code text.

The commit shipped tests but no §11.4.135 RED/GREEN polarity guard mechanically distinguishing the pre-fix from post-fix render path. This work closes that governance gap.

## The guard

dashboard/src/screens/FleetScreen.health-regression.test.tsx — one RED\_MODE polarity switch (default 0):

- **RED\_MODE=1** — reproduces the PRE-FIX render path verbatim (<dd>{health as React.ReactNode}</dd> with the *real* object-shaped health) and asserts React **does throw** /Objects are not valid as a

React child/. This is the §11.4.115 RED-on-the-broken-shape proof that the defect is real and the guard genuinely catches it.

- **RED\_MODE=0** (default — the standing GREEN guard) — renders the CURRENT shipped <DeviceDetail> with the real object-shaped health and asserts it (a) does NOT crash, (b) projects the object to text (“ok” / “unhealthy” + last\_error\_code), (c) never leaks "[object Object]" into the DOM, plus a negative-shape (ok: false) case and a compile-time type-contract lock.

If a future regression re-types health to a scalar or re-renders the object directly, the GREEN assertions FAIL.

## Commands run + results (full transcript: vitest\_run.txt)

| Command                                                                           | Result                                                             |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------|
| npx vitest run src/screens/FleetScreen.health-regression.test.tsx (GREEN default) | <b>3 passed</b>                                                    |
| RED_MODE=1 npx vitest run src/screens/FleetScreen.health-regression.test.tsx      | <b>1 passed</b> — the throw was asserted; pre-fix crash reproduced |
| npx vitest run (full suite)                                                       | <b>58 passed</b> (was 55 at commit 4658125; +3 new guards)         |
| npm run typecheck                                                                 | <b>clean</b> (tsc app + node + vitest tsconfigs)                   |

Tooling: node v22.22.3, npm 10.9.8, Vitest 2.1.9, node\_modules already installed from the committed package-lock.json.